

¿Qué es Suricata?

Web <https://suricata.io/download/>

- **Suricata** es un **IDS/IPS (Sistema de Detección/Prevención de Intrusos)** y motor de análisis de tráfico de red.
- Fue desarrollado por **OISF (Open Information Security Foundation)**.
- Sus principales funciones:
 - Inspeccionar tráfico en tiempo real.
 - Detectar amenazas en base a **firmas (rules)**.
 - Analizar protocolos (HTTP, DNS, TLS, FTP, SSH...).
 - Generar logs detallados y estadísticas.
 - Puede actuar en modo **IDS (solo detecta)** o **IPS (detecta y bloquea)**.
- Similar a **Snort**, pero más moderno y con soporte nativo de **multihilo**, lo que lo hace más rápido en redes grandes.

◆ ¿Qué es Suricata con Machine Learning?

- Suricata normalmente se basa en **firmas** (reglas predefinidas) → detecta ataques conocidos.
- Con **Machine Learning**:
 - Se pueden detectar **anomalías en el tráfico** sin depender de reglas exactas.
 - Se usa para identificar **ataques “zero-day”** o comportamientos sospechosos.
 - Normalmente se integra con:
 - **ELK (Elasticsearch, Logstash, Kibana)** para recolectar y visualizar datos.
 - **ML frameworks (Scikit-learn, TensorFlow, PyTorch)** para entrenar modelos sobre los logs.
 - Ejemplo: detectar patrones anormales de uso de DNS, picos de tráfico inusuales, o intentos de exfiltración de datos.

👉 En FP, es más didáctico usar Suricata en su forma básica (con reglas) y luego **explicar que ML se aplica sobre sus logs**.

◆ Ejercicios sencillos de Suricata en Kali Linux para alumnos

📌 Supongamos que ya tienen Kali instalado.

1 Instalar Suricata

```
sudo apt update
```

```
sudo apt install suricata -y
```

Ver versión instalada:

```
suricata --build-info
```

```
jcsantos at parrot in ~
└─$ sudo suricata --build-info
This is Suricata version 6.0.10 RELEASE
Features: NFQ PCAP_SET_BUFF AF_PACKET HAVE_PACKET_FANOUT LIBCAP_NG LIBNET1.1 HAVE_HTTP_URI_NORMALIZE_HOOK PCRE_JIT HAVE_NSS HAVE_LUA HAVE_LUAJIT HAVE_LIBJANSSON TLS TLS_C11 MAGIC RUST
SIMD support: none
Atomic intrinsics: 1 2 4 8 byte(s)
64-bits, Little-endian architecture
GCC version 12.2.0, C version 201112
compiled with _FORTIFY_SOURCE=2
L1 cache line size (CLS)=64
thread local storage method: _Thread_local
compiled with LibHTTP v0.5.42, linked against LibHTTP v0.5.42
```

2 Ejecutar Suricata en modo IDS con interfaz de red

```
sudo suricata -i eth0
```

👉 Sustituir eth0 por la interfaz real (pueden verlas con ip a).

```
jcsantos at parrot in ~
└─$ sudo suricata -i ens33
27/1/2026 -- 09:33:54 - <Notice> - This is Suricata version 6.0.10 RELEASE running in SYSTEM mode
27/1/2026 -- 09:33:54 - <Warning> - [ERRCODE: SC_ERR_NO_RULES(42)] - No rule files match the pattern /etc/suricata/rules/suricata.rules
27/1/2026 -- 09:33:55 - <Notice> - all 4 packet processing threads, 4 management threads initialized, engine started.
```

3 Ver los logs de tráfico

Suricata genera logs en:

```
cd /var/log/suricata/
```

```
ls
```

Archivos clave:

- fast.log → alertas rápidas.
- eve.json → logs detallados en JSON

```
jcsantos at parrot in /var/log/suricata
└─$ cat fast.log
01/27/2026-09:22:04.376814  [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.62:8
-> 192.168.0.79:0
01/27/2026-09:22:04.376898  [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.79:0
-> 192.168.0.62:0
01/27/2026-09:22:04.404549  [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.115:8
-> 192.168.0.79:0
01/27/2026-09:22:04.404644  [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.79:0
```

```
jcsantos at parrot in /var/log/suricata
└─$ cat eve.json
{"timestamp": "2026-01-27T08:55:50.580168+0100", "event_type": "stats", "stats": {"uptime": 8, "capture": {"kernel_packets": 30, "kernel_drops": 0, "errors": 0}, "decoder": {"pkts": 32, "bytes": 8047, "invalid": 0, "ipv4": 16, "ipv6": 7, "ethernet": 32, "chdlc": 0, "raw": 0, "null": 0, "small": 0, "tcp": 0, "udp": 21, "sctp": 0, "icmpv4": 0, "icmpv6": 2, "ppp": 0, "pppoe": 0, "geneve": 0, "gre": 0, "vlan": 0, "vlan_qinq": 0, "vxlan": 0, "vntag": 0, "ieee8021ah": 0, "teredo": 0, "ipv4_in_ipv6": 0, "ipv6_in_ipv6": 0, "mpls": 0, "avg_pkt_size": 251, "max_pkt_size": 669, "max_mac_addr_src": 0, "max_mac_addr_dst": 0, "erspan": 0, "event": {"ipv4": {"pkt_too_small": 0, "hlen_too_small": 0, "iplen_smaller_than_hlen": 0, "trunc_pkt": 0, "opt_invalid": 0, "opt_invalid_len": 0, "opt_malformed": 0, "opt_pad_required": 0, "opt_eol_required": 0, "opt_duplicate": 0, "opt_unknown": 0, "wrong_ip_version": 0, "icmpv6": 0, "frag_pkt_too_large": 0, "frag_overlap": 0, "frag_ignored": 0}, "icmpv4": {"pkt_too_small": 0, "unknown_type": 0, "unknown_code": 0, "ipv4_trunc_pkt": 0, "ipv4_unknown_ver": 0}, "icmpv6": {"unknown_type": 0, "unknown_code": 0, "pkt_too_small": 0, "ipv6_trunc_pkt": 0, "mld_message_with_invalid_hl": 0, "unassigned_type": 0, "experimentation_type": 0}, "ipv6": {"pkt_too_small": 0, "trunc_pkt": 0, "trunc_exthdr": 0, "exthdr_dupl_fh": 0, "exthdr_useless_fh": 0, "exthdr_dupl_rh": 0, "exthdr_": 0}}
```

4 Probar una regla sencilla (detectar ping)

1. Editar reglas locales:

```
sudo nano /etc/suricata/rules/local.rules
```

2. Añadir:

```
alert icmp any any -> any any (msg:"Ping detectado"; sid:1000001; rev:1;)
```

```
jcsantos at parrot in ~ 27a0:0e23:7a10:131 -> ff02::0000:0000:0000:0000:0001:ff
[O] sudo cat /etc/suricata/rules/local.rules
[sudo] password for jcsantos: 0e23:7a10:131 -> ff02::0000:0000:0000:0000:0000:00
alert icmp any any -> any any (msg:"Ping detectado"; sid:1000001; rev:1;)
fe80::0000:0000:0000:0000:6bd6:27a0:0e23:7a10:131 -> ff02::0000:0000:0000:0000:0000:00
jcsantos at parrot in ~ 0001:1] Ping detectado [**] [Classification: (null)] [
0000:0000:0000:0000:6bd6:27a0:0e23:7a10:131 -> ff02::0000:0000:0000:0000:0000:00
35:46:751143 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [
```

3. Probar la regla:

```
sudo suricata -i eth0 -S /etc/suricata/rules/local.rules
```

4. Desde otra máquina, hacer un ping al Kali → debería registrarse en fast.log.

```
jcsantos at parrot in /var/log/suricata
[O] cat fast.log
01/27/2026-09:22:04.376814 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.62:8
-> 192.168.0.79:0
01/27/2026-09:22:04.376898 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.79:0
-> 192.168.0.62:0
01/27/2026-09:22:04.404549 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.115:8
-> 192.168.0.79:0
01/27/2026-09:22:04.404644 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.79:0
-> 192.168.0.115:0
01/27/2026-09:22:04.413902 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.108:8
-> 192.168.0.79:0
01/27/2026-09:22:04.413992 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.0.79:0
-> 192.168.0.108:0
01/27/2026-09:22:34.873322 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {IPv6-ICMP} fe80::0000:
```

5 Analizar un archivo pcap

Descargar un pcap de ejemplo:

```
wget https://www.malware-traffic-analysis.net/2020/08/31/2020-08-31-traffic-analysis-exercise.pcap.zip
```

```
unzip 2020-08-31-traffic-analysis-exercise.pcap.zip
```

Ejecutar Suricata:

```
sudo suricata -r 2020-08-31-traffic-analysis-exercise.pcap -l /tmp/suricata-test
```

Ver alertas:

```
cat /tmp/suricata-test/fast.log
```

6 Ejercicio final (para alumnos)

👉 **Reto en clase:**

- Crear **una regla personalizada** que detecte tráfico HTTP hacia example.com.
 - Probar con:
 - `curl http://example.com`
 - Comprobar en fast.log que la alerta salta.
-

✓ Resumen para FP

- **Nivel básico:** instalación, ejecución, logs y reglas simples.
- **Nivel intermedio:** análisis de PCAPs y reglas personalizadas.
- **Nivel avanzado (explicación):** integración con Machine Learning para detectar anomalías en tráfico.